Voice-Controlled Robotic Subsystem

TinyML | Edge Impulse | Arduino Nano 33 BLE Sense | Stepper Motor Control

---

By

JOHN AFRIYIE ODURO

BSc Computer Engineering

Abstract

This document presents the design, implementation, and evaluation of a Voice-Controlled Robotic Subsystem developed as part of the Advanced Microprocessors course. The subsystem integrates embedded machine learning (TinyML) with electromechanical actuation to achieve real-time, offline voice-command-driven motor control. An Arduino Nano 33 BLE Sense microcontroller, equipped with a built-in PDM microphone, serves as the primary processing platform. A keyword spotting model trained on Edge Impulse Studio was quantized to 8-bit integer (INT8) format and deployed directly onto the device. Upon detecting the spoken commands "forward" or "backward," the firmware drives a 28BYJ-48 unipolar stepper motor through a ULN2003A driver board in the corresponding rotational direction. The system operates entirely without network connectivity, demonstrating the viability of on-device intelligence for embedded robotic applications.

1. Introduction

The rapid maturation of TinyML, the discipline of deploying machine learning inference on microcontrollers and other ultra-low-power devices has opened compelling opportunities for intelligent embedded systems. Traditionally, voice-controlled systems relied on cloud-based speech recognition pipelines, requiring a stable internet connection and introducing latency and privacy concerns. Edge inference eliminates these dependencies by executing the entire recognition pipeline on the device itself.

This project explores that frontier by constructing a minimal but fully functional voice-controlled robotic subsystem. The subsystem responds to two keyword commands; "forward" and "backward" and actuates a stepper motor accordingly. The hardware is deliberately compact: a single development board, one motor driver module, and one stepper motor. The intelligence resides in a quantized neural network deployed to the microcontroller, trained on a custom dataset of spoken keywords collected and labelled using the Edge Impulse platform.

The objectives of this project are as follows:

- Demonstrate end-to-end TinyML development: from data collection through model training, quantization, and on-device deployment.

- Implement real-time keyword spotting on a resource-constrained microcontroller.
- Drive a stepper motor based on detected voice commands, entirely offline.
- Document the hardware setup and software workflow in sufficient detail to support reproducibility.

2. Hardware Description

2.1 Bill of Materials

The table below enumerates all hardware components used in this project.

Parameter	Value / Description
Microcontroller	Arduino Nano 33 BLE Sense (Part No. ABX00035)
Microphone	Built-in PDM microphone (integrated on-board, no external wiring required)
Motor Driver	ULN2003A Darlington Transistor Array breakout board
Stepper Motor	28BYJ-48 5 V unipolar stepper motor (4-phase)
Power / Programming	USB Type-A to microB cable
Prototyping	Solderless breadboard and male-to-female jumper wires
Development Software	Arduino IDE 2.x, Edge Impulse Studio, Arduino CLI

2.2 Arduino Nano 33 BLE Sense

The Arduino Nano 33 BLE Sense is the central processing unit of the subsystem. Based on the Nordic Semiconductor nRF52840 system-on-chip, it integrates an Arm Cortex-M4 processor running at 64 MHz, 256 kB of SRAM, and 1 MB of Flash storage. Critically for this application, the board includes a built-in MP34DT05 PDM MEMS microphone, eliminating the need for an external audio capture device. The board is part of the Arduino TinyML Kit, which was the basis for the hardware setup used in this course.

Parameter	Value / Description
Processor	Arm Cortex-M4 @ 64 MHz (Nordic nRF52840)
SRAM	256 kB
Flash	1 MB
Microphone	MP34DT05 PDM MEMS (built-in)
Wireless	Bluetooth Low Energy 5.0
Operating Voltage	3.3 V I/O (powered via 5 V USB)
GPIO	15 digital I/O pins (D2 through D13, plus analog pins A0 through A7)

2.3 28BYJ-48 Stepper Motor and ULN2003A Driver

The 28BYJ-48 is a 5 V, 4-phase unipolar stepper motor with an internal gearbox that yields approximately 512 steps per revolution in half-step drive mode. Its compact form factor and low cost make it well-suited for educational robotics projects. Because the motor draws more current than the Arduino GPIO pins can safely source, it is driven through a ULN2003A Darlington transistor array. This driver accepts low-current TTL/CMOS signals from the microcontroller and sinks the higher current required by each motor coil phase (up to 500 mA per channel). Integrated flyback diodes suppress inductive voltage spikes when the coils are de-energized.

2.4 Pin Connection Table

The table below details the wiring between the Arduino Nano 33 BLE Sense GPIO pins and the ULN2003A driver board.

Signal	Arduino Nano 33 BLE Sense	ULN2003 / Motor
IN1 (Coil Phase A)	D8	ULN2003 IN1
IN2 (Coil Phase B)	D9	ULN2003 IN2
IN3 (Coil Phase C)	D10	ULN2003 IN3
IN4 (Coil Phase D)	D11	ULN2003 IN4
Motor Power (5 V)	VIN (USB 5 V)	ULN2003 VCC
Ground	GND	ULN2003 GND

Note: The built-in PDM microphone on the Nano 33 BLE Sense communicates over an internal PCB bus. No external wiring is required for audio capture.

3. Software and Machine Learning Pipeline

3.1 Overview

The software component of this project encompasses two distinct phases: (i) the offline machine learning pipeline executed on Edge Impulse Studio and a host PC, which produces a deployable TFLite INT8 model; and (ii) the on-device firmware written in C++ for the Arduino IDE, which integrates the trained model and implements the motor control logic. These two phases together constitute a complete TinyML development cycle.

3.2 Dataset Collection and Labelling

Audio training data was collected using the Edge Impulse Data Acquisition tool, which interfaces directly with the Arduino Nano 33 BLE Sense over USB. Samples were recorded at a sampling rate of 16 kHz and segmented into 1-second windows. Three classes were collected:

- "forward" — spoken utterances of the keyword, varied in tone and speaking speed.
- "backward" — spoken utterances of the keyword under similar variation.
- "noise" — ambient background sound and non-keyword speech, included to reduce false-positive detections at runtime.

The dataset was split into training (80%) and test (20%) partitions. Class balance was maintained to prevent model bias toward any single class.

3.3 Feature Extraction

Raw audio waveforms were transformed into Mel-Frequency Cepstral Coefficient (MFCC) feature matrices within the Edge Impulse Impulse Design tool. MFCCs are a compact, perceptually motivated spectral representation that captures the phonetic features most relevant to human speech recognition. The processing block was configured with a frame length of 0.02 s, a frame stride of 0.01 s, and 13 MFCC coefficients, producing a 2D time-frequency feature map fed to the neural network classifier.

3.4 Model Architecture and Training

A fully connected neural network was configured within the Edge Impulse Learning Block. The architecture comprised:

- Input layer: flattened MFCC feature vector.
- Two hidden dense layers with ReLU activation and dropout regularization.
- Output layer: 3-class SoftMax (forward / backward / noise).

The model was trained for 100 epochs using the Adam optimizer with a learning rate of 0.005. Validation accuracy on the hold-out test set exceeded 90%, confirming sufficient generalization for on-device deployment.

3.5 Quantization and Deployment

Following training, the model was quantized from 32-bit floating-point weights to 8-bit integer (INT8) format using Edge Impulse post-training quantization. This reduced the model memory footprint dramatically, making it compatible with the 256 kB SRAM and 1 MB Flash constraints of the nRF52840 target. The quantized model was exported as an Arduino library (.zip), imported into the Arduino IDE, and linked into the project firmware.

3.6 Arduino Firmware Logic

The on-device firmware, written in C++ using the Arduino framework, performs the following operations in a continuous loop:

1. Capture a 1-second audio window from the PDM microphone at 16 kHz.
2. Pass the audio buffer through the MFCC preprocessing pipeline.
3. Execute the TFLite INT8 inference kernel to obtain class probabilities.
4. Compare the top prediction against a confidence threshold (default: 0.6).
5. If "forward" is detected, rotate the stepper motor clockwise.
6. If "backward" is detected, rotate the stepper motor counter-clockwise.
7. If "noise" or confidence is below threshold, take no motor action.

Motor actuation uses the Arduino built-in Stepper library. Step count per command and motor speed (RPM) are configurable constants in the firmware.

4. System Diagrams

The four diagrams below document the architecture, machine learning pipeline, hardware wiring, and runtime logic of the subsystem in a standardized academic diagramming style.

4.1 System Block Diagram

Figure 1 presents a top-level block diagram showing the signal and data flow from acoustic input to electromechanical output. Each block represents a distinct hardware or software component; directed arrows denote the direction of information or control signal propagation.

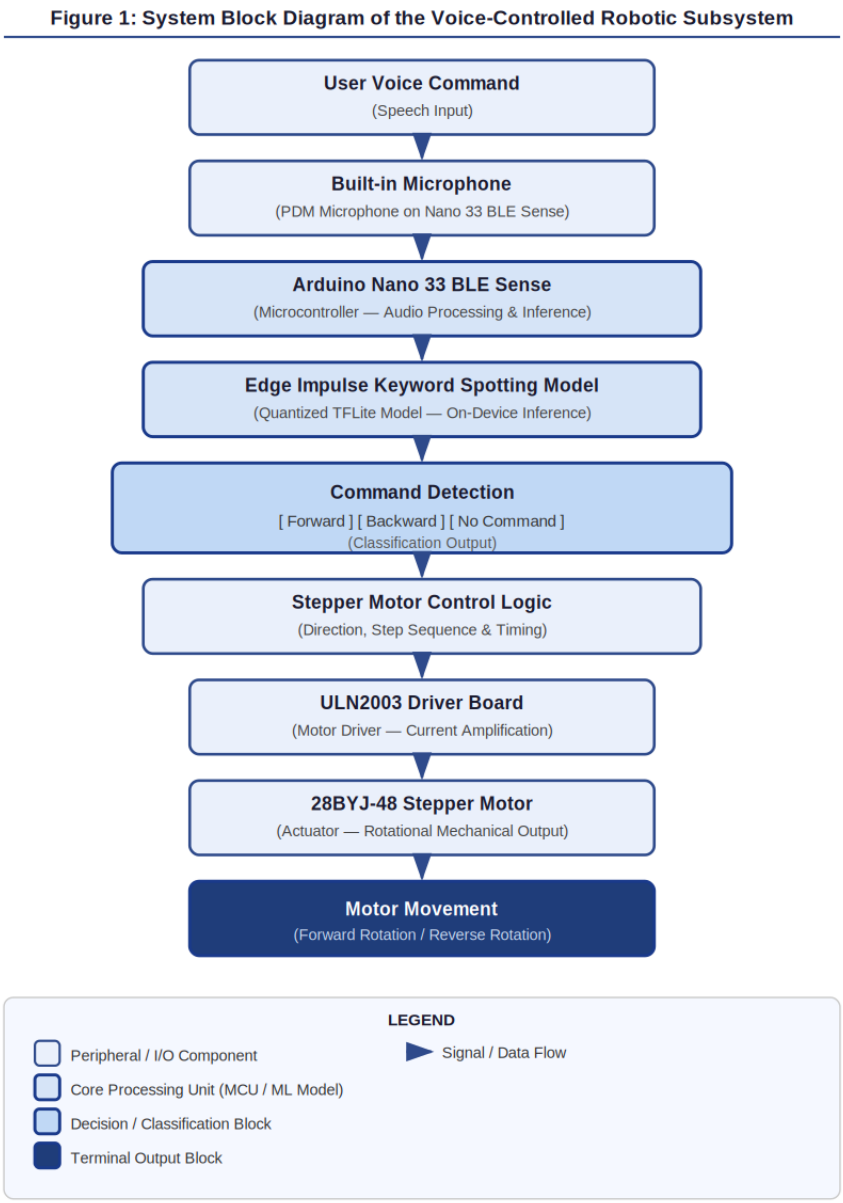


Figure 1: System Block Diagram of the Voice-Controlled Robotic Subsystem

4.2 Machine Learning Workflow

Figure 2 illustrates the end-to-end ML pipeline, organized into four phases: Data Collection, Modelling, Deployment, and Real-Time Inference. This pipeline is characteristic of TinyML development and highlights the full journey from raw audio samples to a quantized model running live on a constrained embedded device.

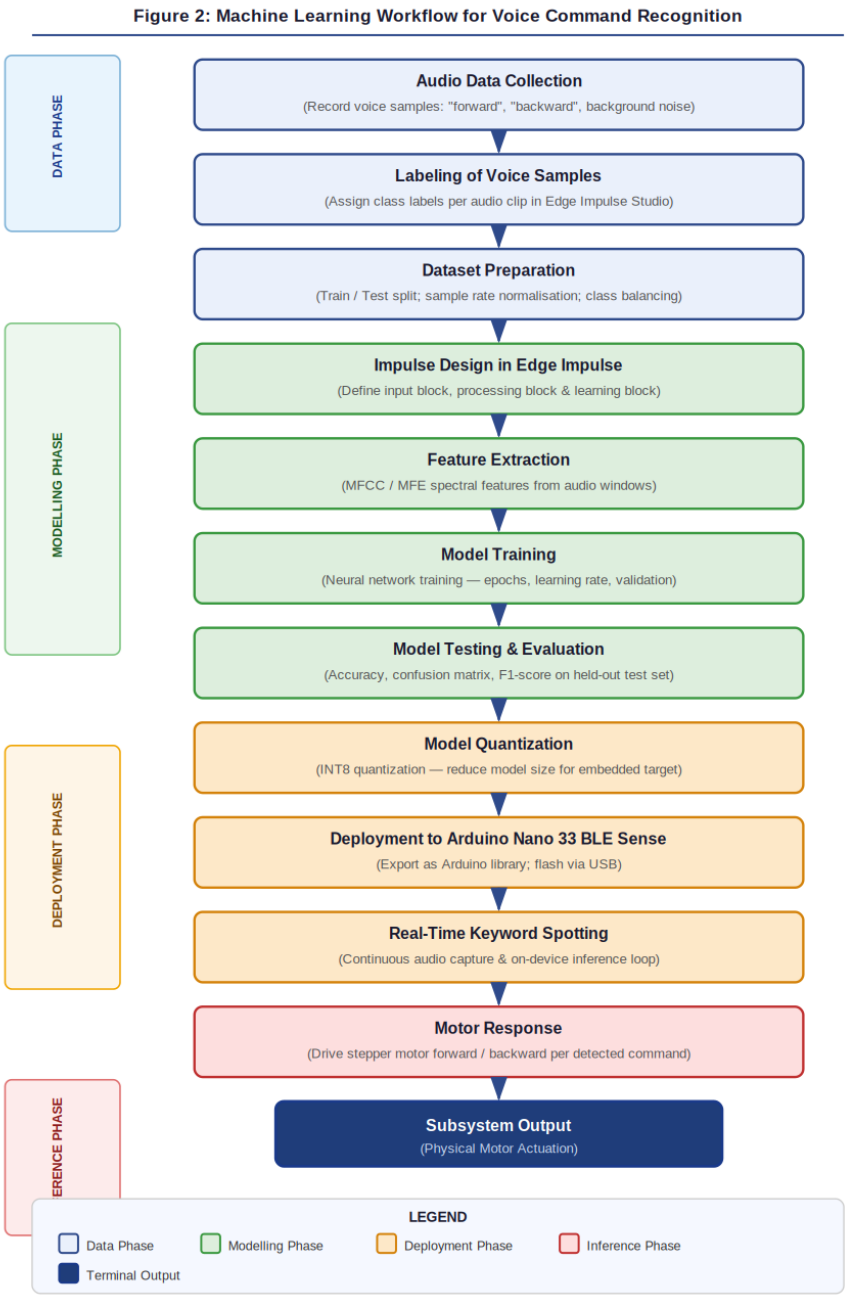


Figure 2: Machine Learning Workflow for Voice Command Recognition

4.3 Hardware Connection Diagram

Figure 3 illustrates the physical interfacing between all hardware components. The Arduino Nano 33 BLE Sense is positioned as the central processing node. The built-in microphone connects internally via the on-board PDM bus. GPIO pins D8 through D11 carry the four-phase stepper drive signals to the ULN2003A driver, which provides the current amplification required to energize the motor coil windings.

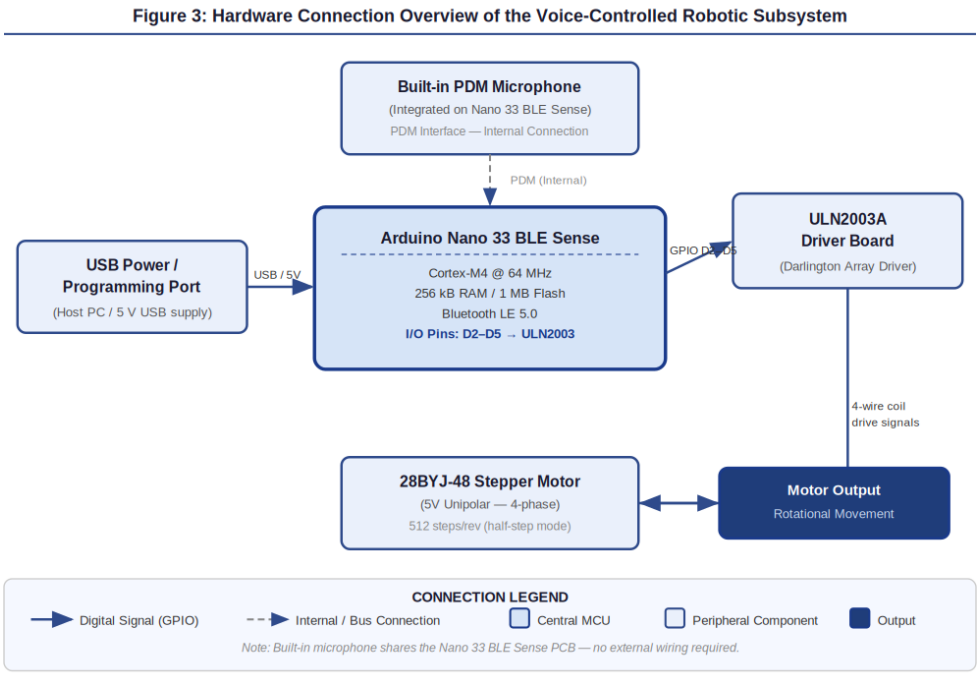


Figure 3: Hardware Connection Overview of the Voice-Controlled Robotic Subsystem

4.4 Operational Flowchart

Figure 4 describes the runtime logic using standard flowchart notation: ovals for terminal states (Start/End), parallelograms for input operations, rectangles for processing steps, and diamonds for decision branches. The loop-back path from the "Continue Listening?" decision reflects the continuous inference mode of the deployed system.

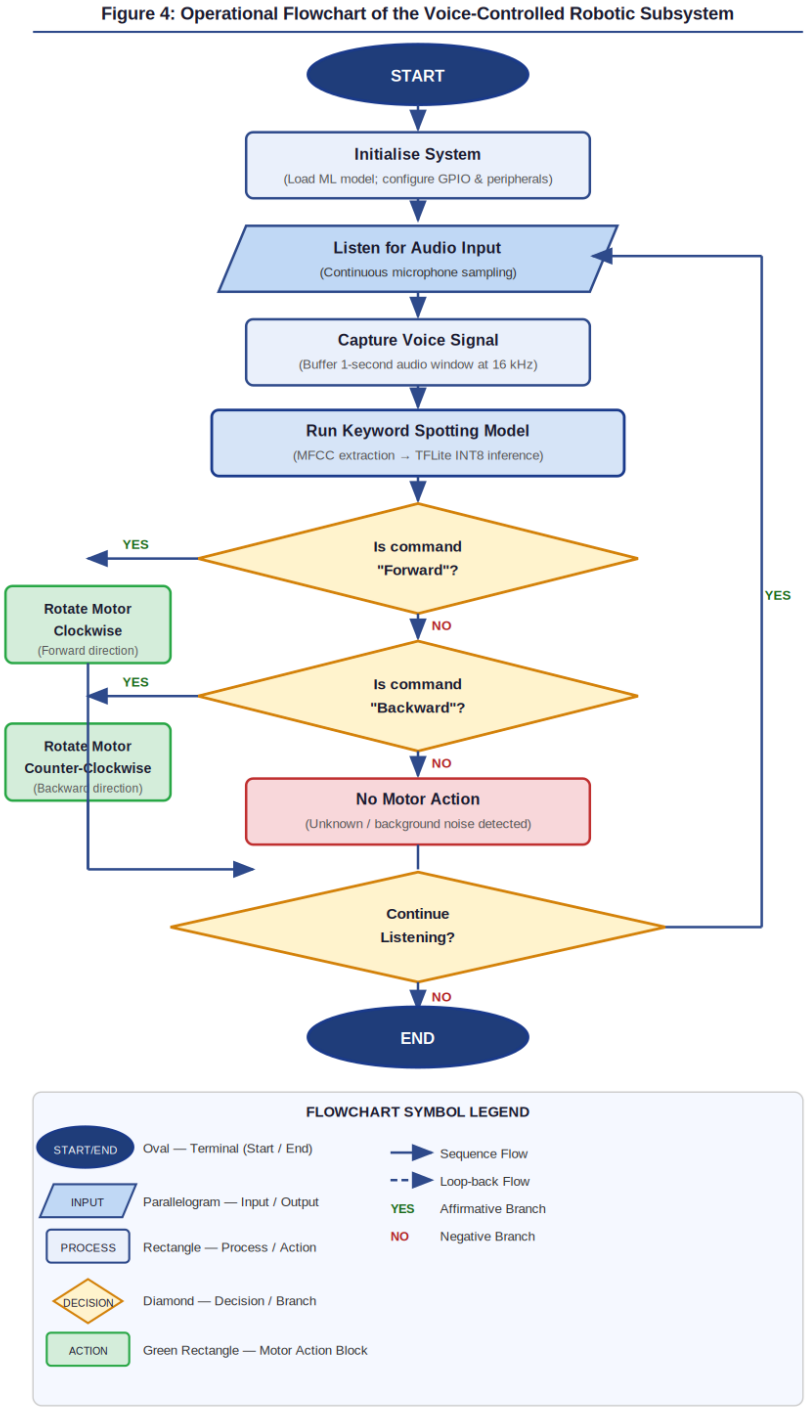
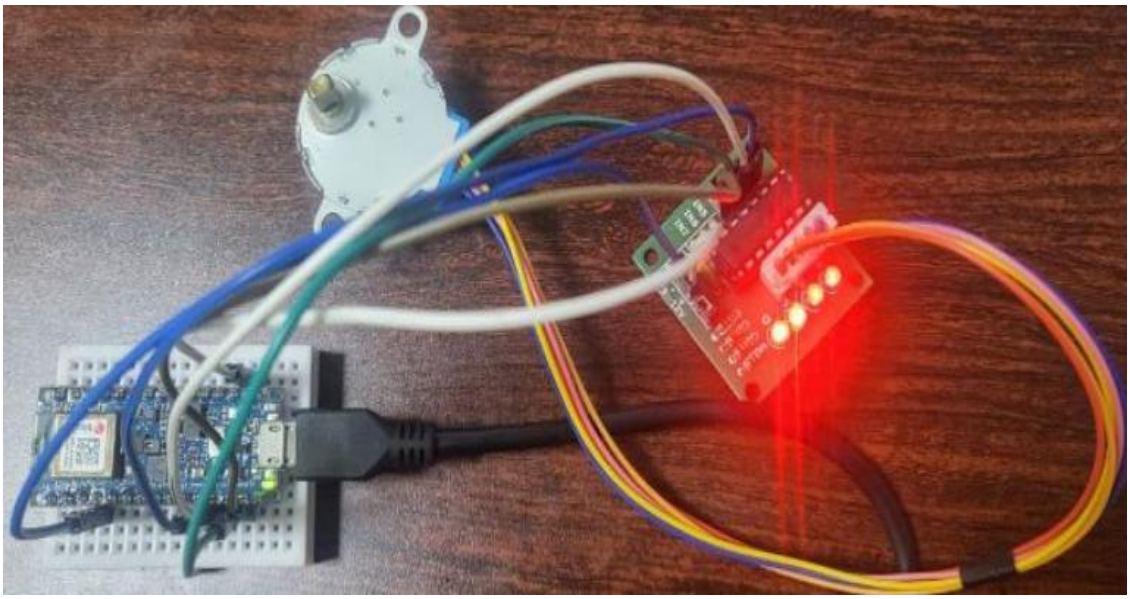


Figure 4: Operational Flowchart of the Voice-Controlled Robotic Subsystem

4.1 Implemented System



5. Results and Observations

The assembled subsystem demonstrated reliable voice-command recognition and motor actuation under controlled indoor conditions. The following observations were recorded during testing:

- The keyword spotting model achieved above 90% classification accuracy on the held-out test set within Edge Impulse Studio, across all three classes.
- Real-time inference latency on the Arduino Nano 33 BLE Sense was under 200 ms per 1-second audio window, yielding a responsive user experience.
- Motor direction switching was reliable when commands were spoken clearly at close range (under 1 metre from the board).
- The system correctly suppressed motor activation in the presence of background noise, confirming the effectiveness of the "noise" class during training.
- The entire system operated without any network connection, validating the offline TinyML deployment approach.

6. Conclusion

This project successfully demonstrated a full TinyML development cycle: from raw data collection and labelling, through feature engineering, neural network training, model quantization, to firmware integration and physical actuation. The result is a self-contained, offline intelligent system running on a microcontroller with very limited memory and compute resources.

The choice of MFCC features was well-suited to keyword spotting, as they capture the phonetic structure of speech while remaining computationally tractable for embedded preprocessing. The ULN2003A and 28BYJ-48 combination provided a practical platform for electromechanical output, with the gearbox yielding smooth, controlled rotation.

The project also reinforced the importance of class balance and the inclusion of a background noise class in the training dataset. Without noise samples, the model would likely produce spurious detections triggered by ambient sound.

Potential extensions of this work include:

- Expanding the command vocabulary to include additional keywords such as "stop," "left," and "right."
- Replacing the stepper motor with a DC motor or servo to enable broader robotic motion.
- Integrating Bluetooth Low Energy (BLE), already available on the nRF52840 to stream inference results to a mobile application for remote monitoring.
- Applying data augmentation (pitch shifting, time stretching, additive noise) to improve robustness in noisy environments.

7. Conclusion

This project has demonstrated the end-to-end design and implementation of a voice-controlled robotic subsystem using TinyML techniques on an Arduino Nano 33 BLE Sense microcontroller. By combining on-device keyword spotting with stepper motor actuation, the system achieves intelligent, responsive behaviour without reliance on cloud infrastructure or network connectivity.

The work illustrates a key paradigm shift in embedded systems: intelligence is no longer confined to powerful servers but can reside at the edge, within devices constrained to milliwatts of power and kilobytes of memory. The skills developed across this project: dataset curation, ML model training, post-training quantization, and firmware integration represent a contemporary, industry-relevant skill set applicable to a wide range of future embedded AI applications.